PHP and Web Integration

Build interactive, secure web applications

Outline

- PHP Built-In Functions
- Session and Cookies
- File Handling in PHP
- Basic Security Considerations (input validation, sanitation
- Introduction to Javascript

Overview

- Integrative Programming focuses on how different web technologies work together to build functional, interactive, and secure web applications. In real-world systems, a website is more than just pages—it must remember users, store data, protect itself from attacks, and provide a smooth user experience.
- In this lecture, you will learn how:
  - PHP manages state and data using sessions, cookies, and files
  - Security practices protect applications from invalid or malicious input
  - JavaScript enhances usability on the client side

PHP Built-in Functions

PHP Built-In Functions

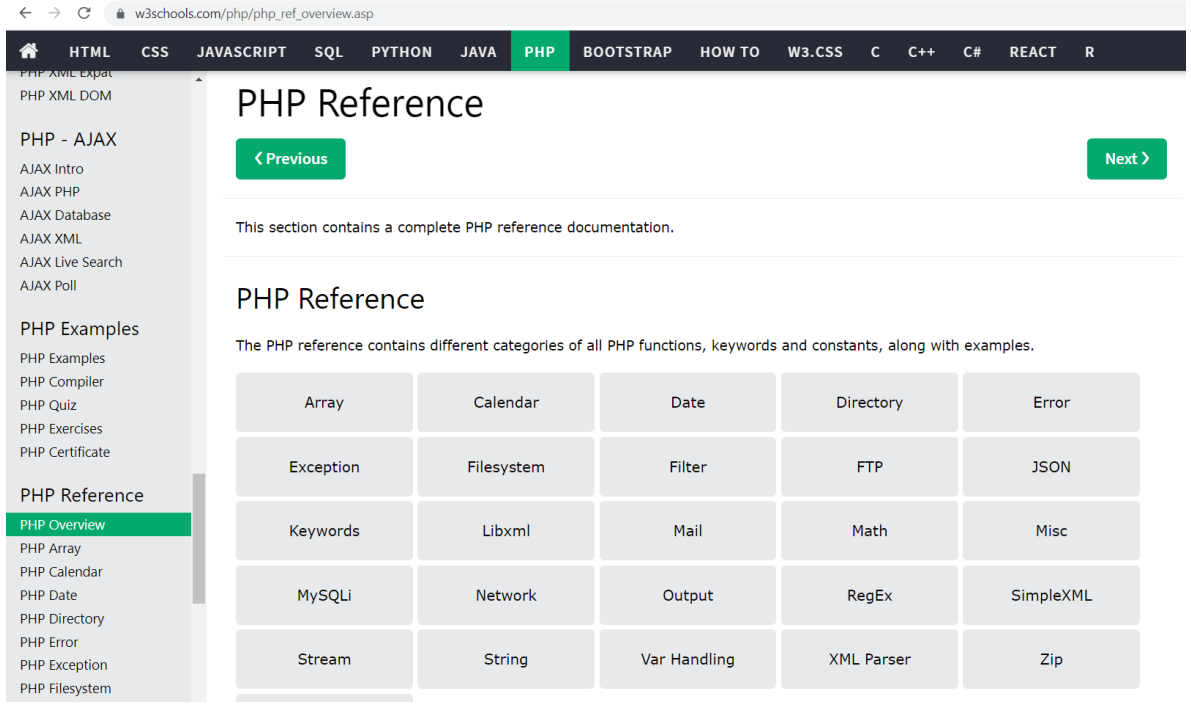
- PHP has lots of built-in functions that we use all the time.
- Much of the power of PHP comes from its built-in functions.
- Many are modeled after C string library functions (i.e. strlen()).

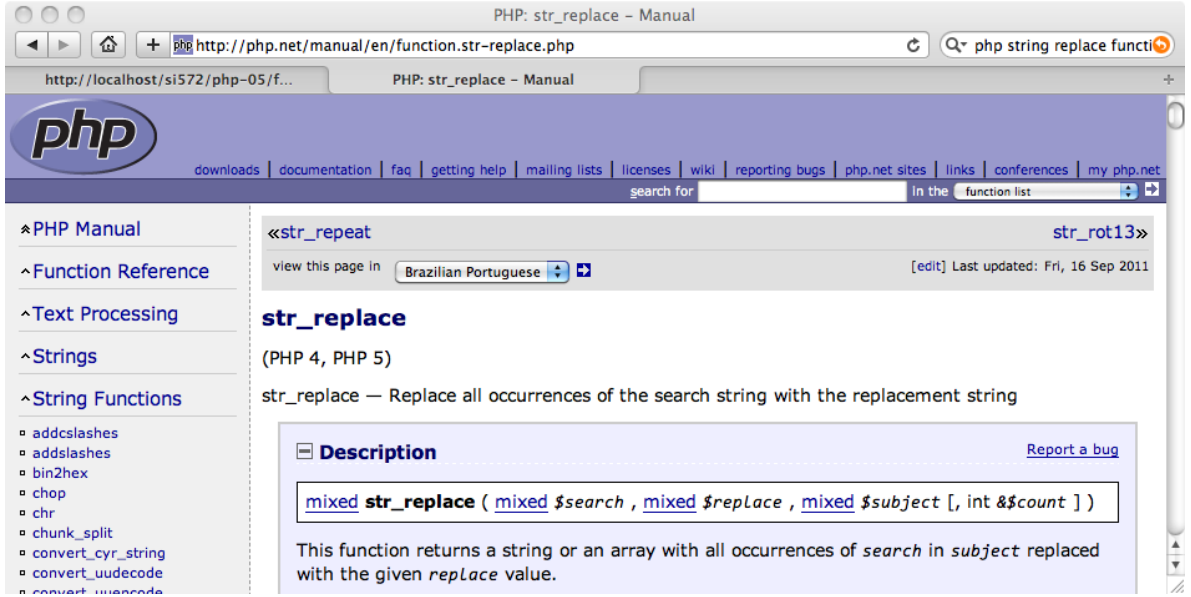
```
echo strrev(" .dlrow olleH");
echo str_repeat("Hip ", 2);
echo strtoupper("hooray!");
echo strlen("intro");
echo "\n";
```

Output

```
Hello world.
Hip Hip
HOORAY!
5
```

PHP Built-in Functions





Using Variable Built-in Functions

- Testing and setting variable types:
  - **`string gettype(mixed variable)`**
    Returns one of the strings: “bool”, “int”, “double”, “string”, “array”, “object”, “resource”, “NULL”, or “unknown type”.
    “Mixed” – is a pseudo-type / signifies that variables of many (or any) data types are accepted as arguments; or function is “oveloaded”
  - **`bool settype(mixed variable, string type)`**
    Sets the type of “variable” to the new type passed as a second argument.
    “Type” can be “bool”, “int”, “double”, “string”, “array”, “object”, “resource”, “NULL”.

```
$a = 56;
echo gettype($a).'<br/>';      // displays int
settype($a, 'double');
echo gettype($a).'<br/>';      // displays double
```

Using Variable Functions

- Specific type-testing functions, return true of false
  - is_array(variable)
  - is_double(variable) is_float(variable) is_real(variable)
  - is_long(variable) is_int(variable) is_integer(variable)
  - is_string(variable)
  - is_bool(variable)
  - is_object(variable)
  - is_resource(variable)
  - is_null(variable)
  - is_scalar(variable) – checks whether variable is an integer, boolean, string or float
  - is_numeric(variable) – checks whether variable is any kind of number or a numeric string
  - is_callable(variable) – checks whether variable is the name of a valid function

Using Variable Functions

- Testing variable status:
  - **`bool isset(mixed variable1 [, mixed variable2 …] )`**
    Returns true if all variables passed to the function exist and false otherwise
  - **`void unset(mixed variable1 [, mixed variable2, …])`**
    Gets rid of the variable(s) it is passed
  - **`bool empty(mixed variable)`**
    Checks if variable exists <u>and</u> has a nonempty, nonzero value

  - These functions are very useful for checking (server-side) if the user filled out the appropriate fields in the form; example:

```
echo isset($tireqty);    // always true for a textbox field
echo empty($tireqty);  // depends on user input, false if form field is empty
```

User-Defined Functions

User-Defined Functions

- We use the function keyword to define a function, we name the function and take optional argument variables. The body of the function is in a block of code { }

```
function greet() {
   print "Hello\n";
}

greet();
greet();
```

Output:

```
Hello
Hello
```

Choosing Function Names

- Much like variable names - but do not start with a dollar sign
- Start with a letter or underscore - consist of letters, numbers, and underscores (_)
- Avoid built-in function names
- Case does not matter – but please do not take advantage of this

Return Values

- Frequently, a function will take its parameters, perform some calculations, and return a result that can be utilized as the calling expression's value for the function call. For this, the return keyword is employed.

```
function greeting() {
    return "Hello";
}

print greeting() . " World\n";
print greeting() . " Universe\n";
```

Output:

```
Hello World
Hello Universe
```

Arguments

- Functions can choose to accept optional arguments. Within the function definition the variable names are effectively "aliases" to the values passed in when the function is called.

```
function greet($lang) {
    if ( $lang == 'es' ) return "Hola";
    if ( $lang == 'fr' ) return "Bonjour";
    return "Hello";
}

print greet('es') . " World\n";
print greet('fr') . " Universe\n";
```

Output:

```
Hola World
Bonjour Universe
```

Optional Arguments

- Arguments can have defaults, and so can be omitted.

```
function greet($lang='es') {
    if ( $lang == 'es' ) return "Hola";
    if ( $lang == 'fr' ) return "Bonjour";
    return "Hello";
}

print greet() . " World\n";
print greet('fr') . " Universe\n";
```

Output:

```
Hola World
Bonjour Universe
```

Call By Value

- The argument variable within the function is an “alias” to the actual variable.
- But even further, the alias is to a *copy* of the actual variable in the function call.

```
function double($alias) {
    $alias = $alias * 2;
    return $alias;
}
$val = 10;
$dval = double($val);
echo "Value = $val Doubled = $dval\n";
```

```
Value = 10 Doubled = 20
```

Call By Reference

- Sometimes we want a function to change one of its arguments, so we indicate that an argument is "by reference" using (&).

```
function triple(&$realthing) {
    $realthing = $realthing * 3;
}

$val = 10;
triple($val);
echo "Triple = $val\n";
```

```
Triple = 30
```

Scope of a Variable in Functions

- In general, variable names used inside of function code do not mix with the variables outside of the function to avoid unexpected side effects if two programmers use the same variable name in different parts of the code.
- We call this “name spacing” the variables. The function variables are in one “name space” whilst the main variables are in another “name space”.

Normal Scope (Isolated)

```
function tryzap() {
    $val = 100;
}

$val = 10;
tryzap();
echo "TryZap = $val\n";
```

```
TryZap = 10
```

Global Scope (Shared)

```
function dozap() {
    global $val;
    $val = 100;
}

$val = 10;
dozap();
echo "DoZap = $val\n";
```

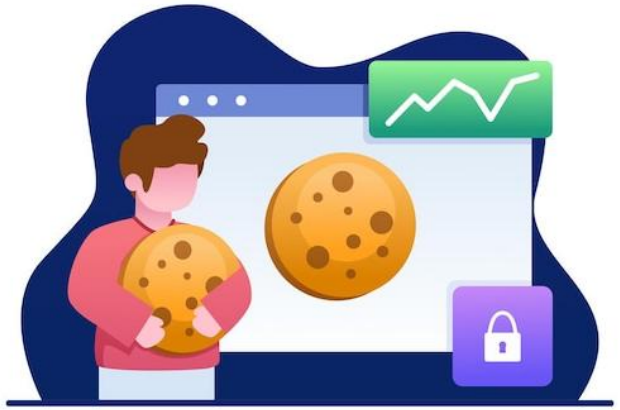
```
DoZap = 100
```

Sessions and Cookies

Sessions and Cookies (1/5)

What are Cookies?

- Cookies are small pieces of text-based data stored in the user's web browser by a website.
- They are automatically sent back to the server with every HTTP request to the same domain.
- Cookies help the server remember information about the user even though HTTP is stateless.
- Cookies can store:
  - User preferences (language, theme)
  - Non-sensitive identifiers (username, visit count)
  - Session IDs (links browser to server-side session data)



Sessions and Cookies (2/5)

Important Characteristics of a Cookie

- Stored on the client side
- Have an expiration time
- Can be viewed, modified, or deleted by the user
- Cookies should never store sensitive data like passwords or personal information.

⚠ Cookies can be modified by users → **not secure for sensitive data**.

Creating a Cookie

```
<?php
setcookie("username", "Juan", time() + 3600); // 1 hour
?>
```

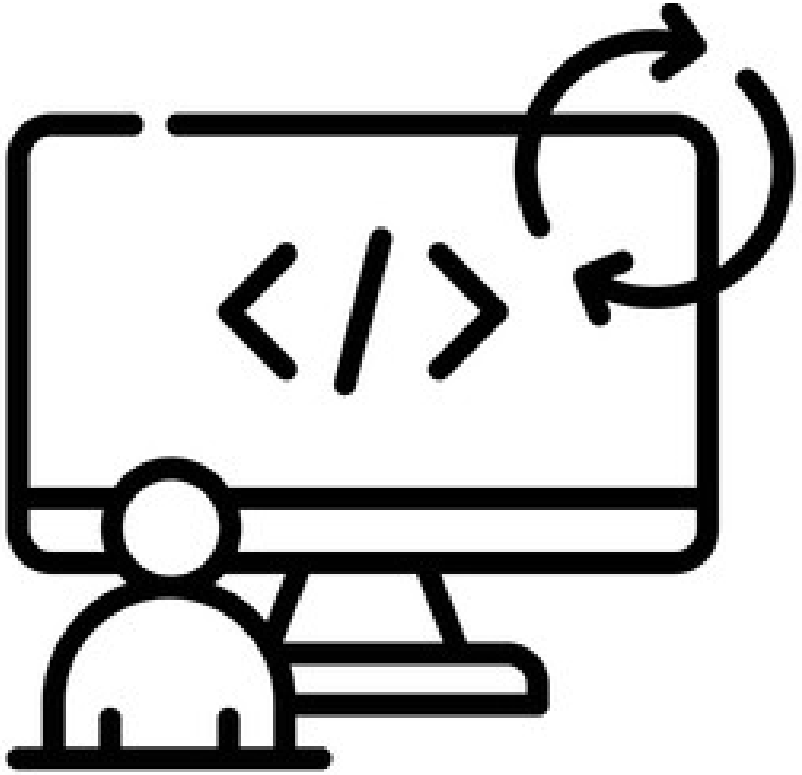
Reading a Cookie

```
<?php
echo $_COOKIE['username'];
?>
```

Sessions and Cookies (3/5)

What are Sessions?

- Sessions are a way to store user-specific data on the server across multiple requests.
- When a session starts, the server generates a unique session ID.
- This session ID is usually stored in a cookie on the browser.
- Actual session data remains safely on the server.



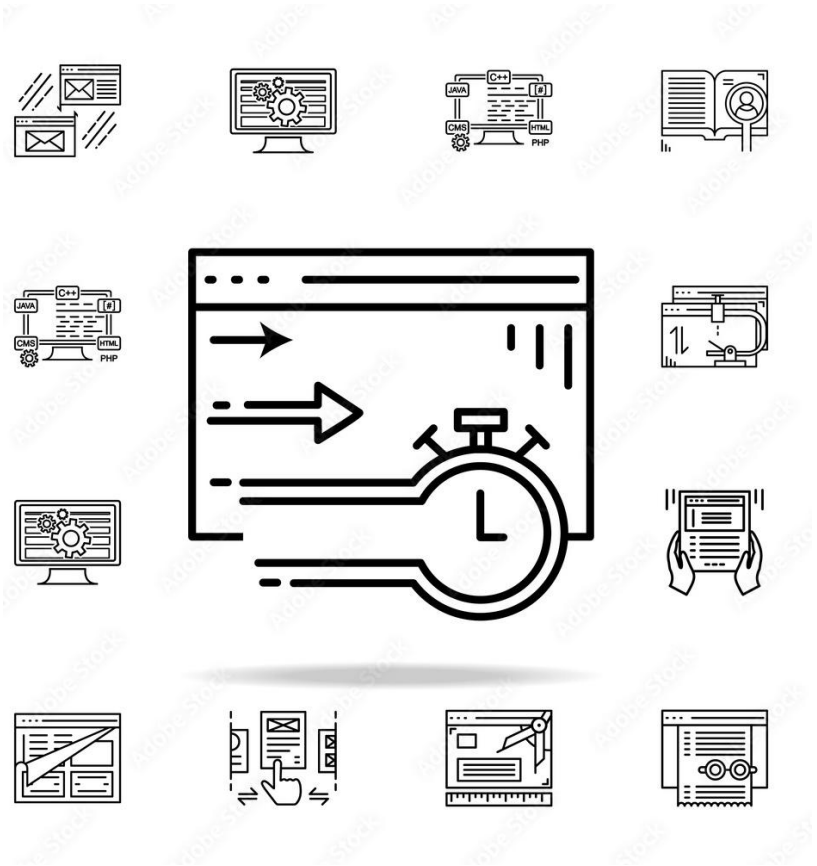
Sessions and Cookies (4/5)

Why Sessions are important?

- HTTP does not remember users by default
- Sessions allow applications to track:
  - Logged-in users
  - User roles and permissions
  - Temporary data like shopping carts

Security Advantage

- Users cannot directly access or modify session data
- More secure than cookies for authentication and sensitive information
- Sessions are ideal for **login systems and protected pages**.



Sessions and Cookies (5/5)

Starting a Session

```
<?php
session_start();
$_SESSION['user'] = "Juan";
?>
```

Accessing Session Data

```
<?php
session_start();
echo $_SESSION['user'];
?>
```

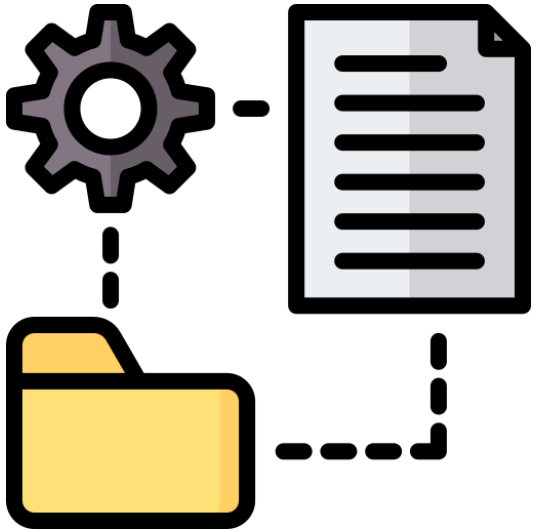
Ending a Session

```
<?php
session_start();
session_destroy();
?>
```

File Handling in PHP

What is File Handling?

- File handling refers to the ability of a program to **create, read, update, and delete files** on the server.
- In PHP, file handling allows web applications to store data permanently without using a database.
- Files are commonly used for:
  - Activity logs
  - Simple records
  - Configuration settings
  - Uploading and storing user files



File Handling – Why It Matters?

- Helps understand how data persistence works
- Introduces server-side resource management
- Serves as a foundation before learning databases
- File handling is best for **small-scale or simple applications**.

Writing to a file

```
<?php
$file = fopen("data.txt", "a");
fwrite($file, "New record added\n");
fclose($file);
?>
```

Reading from a file

```
<?php
$file = fopen("data.txt", "r");
while (!feof($file)) {
echo fgets($file) . "<br>";
}
fclose($file);
?>
```

Basic Security Considerations

Why Security Matters

- Web applications accept input directly from users, which makes them vulnerable to attacks.
- Malicious users can submit harmful data to:
  - Steal information
  - Alter website behavior
  - Damage server resources

Common Security Threats

- SQL Injection – injecting harmful database queries
- Cross-Site Scripting (XSS) – injecting scripts into webpages
- Form tampering – bypassing expected input rules
- **Key Principle:** Never trust user input.

Input Validations

- Input validation ensures that **data follows the expected format before processing**.
- It checks:
  - Data type (number, string, email)
  - Required fields
  - Value range or length

Purpose of Validation:

  - Prevents incorrect or meaningless data
  - Improves data accuracy
  - Reduces security risks early

Example: Ensuring age is numeric or email has correct format

Input Sanitation

Input sanitization **cleans input by removing or escaping harmful characters**.

- Focuses on making data safe for output or storage
- Prevents scripts and HTML from being executed

Purpose of Sanitization:

- Protects users from XSS attacks
- Ensures displayed data is safe

Key Difference:

- Validation checks *if input is acceptable*
- Sanitization makes *input safe*

Introduction to JavaScript

Optional Reinforcement

What is JavaScript?

- JavaScript is a **client-side programming language** that runs directly in the user's browser.
- It allows web pages to respond to user actions **without reloading the page**.
- JavaScript is commonly used for:
  - Form validation
  - Interactive buttons and menus
  - Dynamic content updates

JavaScript in Integrative Programming

- JavaScript is another programming language of HTML and the Web
- JavaScript is client-side scripting language
- JavaScript file have extension “.js”
- It allows you to make things happen in the user’s browser without sending messages back and forth to the server
- Used to validate user data on client side (browser)
- Works along side with PHP, and not as a replacement.

Sample Code: JavaScript (Inline)

```
<!DOCTYPE html>
<html>
<head>
<script type="text/javascript">
 function showMessage() {
 alert("Hello from JavaScript!");
}
</script>
</head>
<body>
<h3>This is a sample JavaScript interactivity</h3>

<button onclick="showMessage()">Click Me</button>

</body>
</html>
```

PHP with JavaScript

```
<!DOCTYPE html>
<html>
<head>
<?php
if (isset($_POST['username'])) {
    $name = $_POST['username'];
    echo "<script type='text/javascript'>
            alert('Welcome, $name!');
          </script>";
}
?>
<form method='post' action='<?php $_SERVER["PHP_SELF"]; ?>'>
  Enter Name: <input type="text" name="username" required>
  <button type="submit">Submit</button>
</form>
</body>
</html>
```

PHP and Web Integration

Build interactive, secure web applications